



WELCOME

**IN403 – Deep Learning and Artificial
Intelligence**

UPDATED: JUNE 2nd, 2018

Overview

INTRODUCTION TO DEEP LEARNING AND NEURAL NETWORKS

In this unit, you will learn about the fundamentals of neural networks and deep learning. You will learn what each is, what algorithms are used, and the applications of the models.

After completing this unit, you should be able to:

- Report on deep learning algorithms.
- Write about neural networks.

Course Outcomes practiced in this unit:

IN403-1: Create deep neural networks for application to the field of information technology.

GEL-1.02: Demonstrate college-level communication through composition of original materials in Standard English

Unit 1

- Be sure to review Unit 1 in Brightspace for more details
- Reading
 - *See next slide*
- Discussion Question (20 points)
 - See following slides.
- Assignment (60 points)
 - See following slides.
- Seminar (20 points)
 - Option 1: Attend each week.
 - Option 2: Reflective Assignment.

Reading

Read, or review, the following in *Python Deep Learning*. To access the texts for this course, navigate to the Library in the Academic Tools section of this course then select "Required Readings."

- Chapter 2: "Neural Networks"
- Chapter 3: "Deep Learning Fundamentals"

Reading Summary

In this unit, you will read about neural networks and deep learning fundamentals. For neural networks, you will read about the three major characteristics and the make up of a neural net. This will help you understand how neural networks can be used for linear and logistic regression and back-propagation. You will also read what deep learning is and how it is used. Finally, you will explore deep learning algorithms and applications.

Discussion

Picking a First Model for Hospital No-Show Prediction

Scenario

You are advising a hospital that wants to reduce patient no-shows. You have five years of appointment data (e.g., age, appointment type, prior no-shows, lead time, location, weather). Leadership wants your recommendation for the first model to deploy.

Initial Post (250–350 words)

Choose one model for the hospital's first rollout:

- Option A: Logistic Regression (baseline, very interpretable)
- Option B: Small Neural Network (more flexible, harder to explain)

Initial Post (250–350 words)

Choose one model for the hospital's first rollout:

- Option A: Logistic Regression (baseline, very interpretable)
- Option B: Small Neural Network (more flexible, harder to explain)

Write your response as a short message to the hospital's executive team (non-technical). Address all four items below:

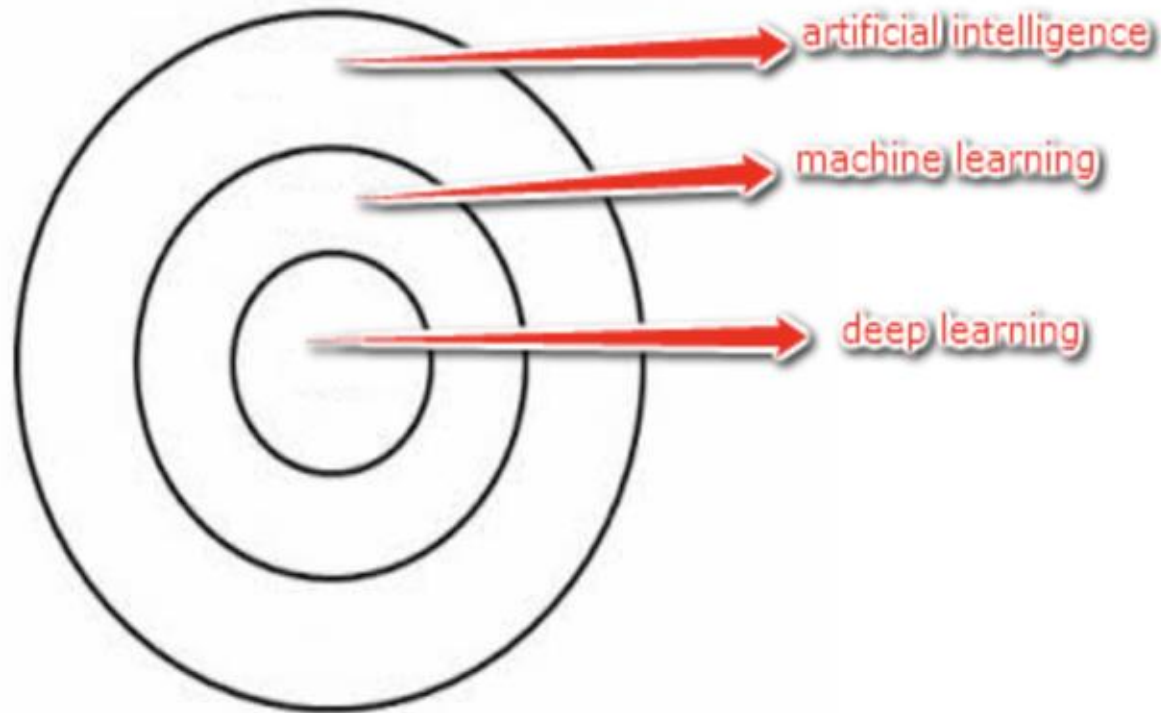
1. Recommendation: State which option you chose (A or B) and give a brief reason.
2. Metric choice: Choose precision or recall as the more important metric for this project and explain why in plain language.
3. Risk control: Name one action you would take to reduce harm if the model is wrong (examples include setting a decision threshold, adding human review for high-risk flags, or monitoring performance monthly).
4. Tradeoff: Identify one drawback of your chosen model (such as lower accuracy or lower explainability) and explain why it is acceptable for a first rollout

Peer Responses

Reply to two classmates. For each reply:

- Ask one “what if” question that challenges or clarifies their recommendation.
- Suggest one concrete improvement (for example, adjusting the threshold, adding monitoring by clinic, or providing a simple explanation summary).

AI vs ML vs DL



Why Neural Networks?

Prior to the introduction of deep neural nets, accuracy had been improving at a slow rate, but after the introduction of deep neural networks, accuracy dropped from an error rate of 40% in 2010 to less than 7% in 2014, and this value is still falling. The human recognition rate is still lower, but only at about 5%.

In addition, deep neural nets "learn" a representation of the data, that is, not only learn to recognize an object, but also learn what the important features that uniquely define the identified object are. By learning to automatically identify features, deep neural nets can be successfully used in unsupervised learning, by naturally classifying objects with similar features together, without the need for laborious human labeling.

Reference:

APA 7th Edition (American Psychological Assoc.)

Zocca, V., Spacagna, G., Slater, D., & Roelants, P. (2017). Python Deep Learning. Packt Publishing.

Parts of Neural Networks

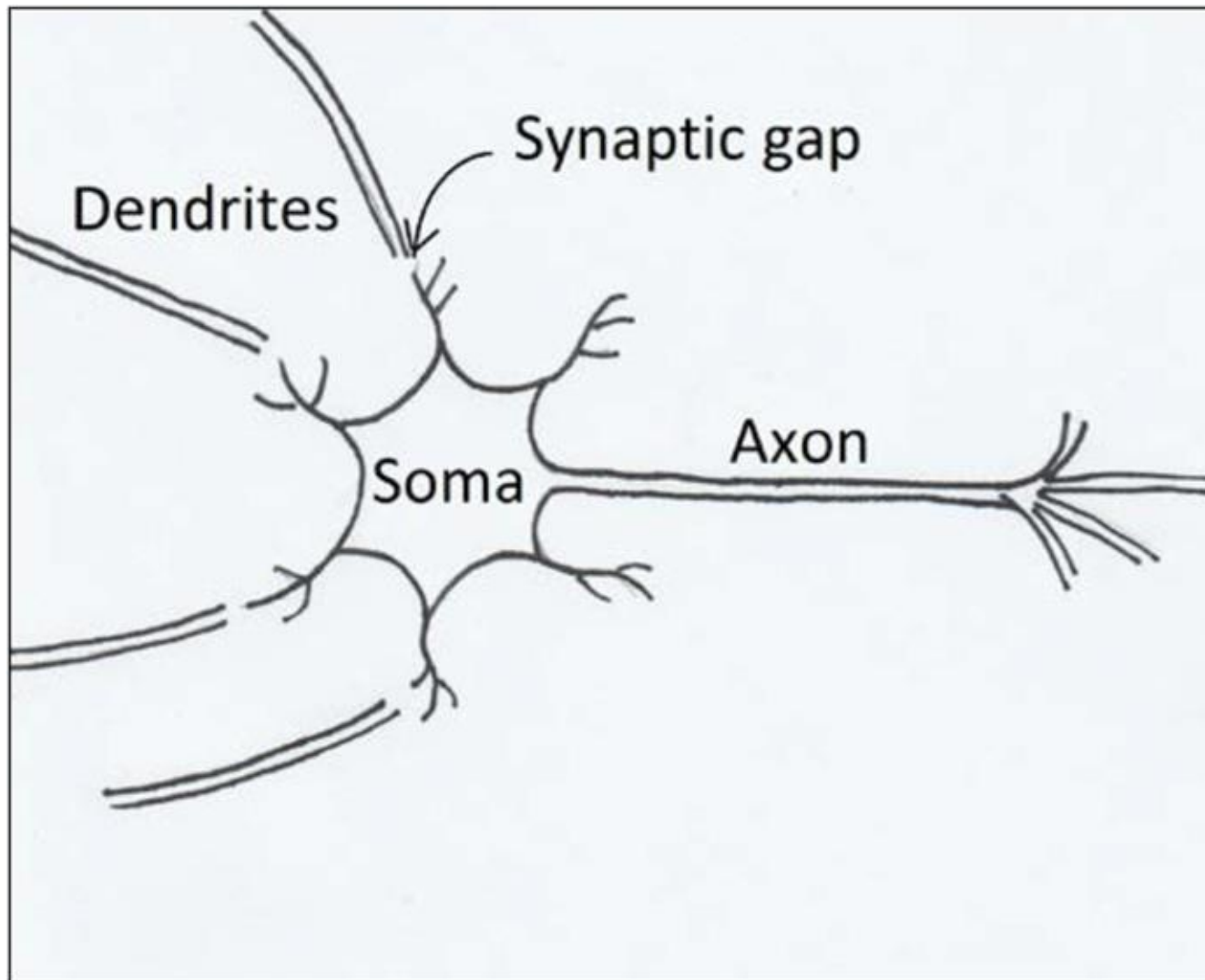
- A neuron is a processing unit that takes an input value and, according to predefined rules, outputs a different value.
- The dendrites bring the input from other neurons to the soma, the neuron's body.
- The soma is where the inputs are processed and summed together. If the input is over a certain threshold, the neuron will "fire" and transmit a single output that is electrically sent through the axons.
- Between the axons of the transmitting neurons and the dendrites of the receiving neurons lies the synaptic gap that mediates chemically such impulses, altering their frequencies.

Reference:

APA 7th Edition (American Psychological Assoc.)

Zocca, V., Spacagna, G., Slater, D., & Roelants, P. (2017). Python Deep Learning. Packt Publishing.

Neuronal Networks



Artificial Neural Networks

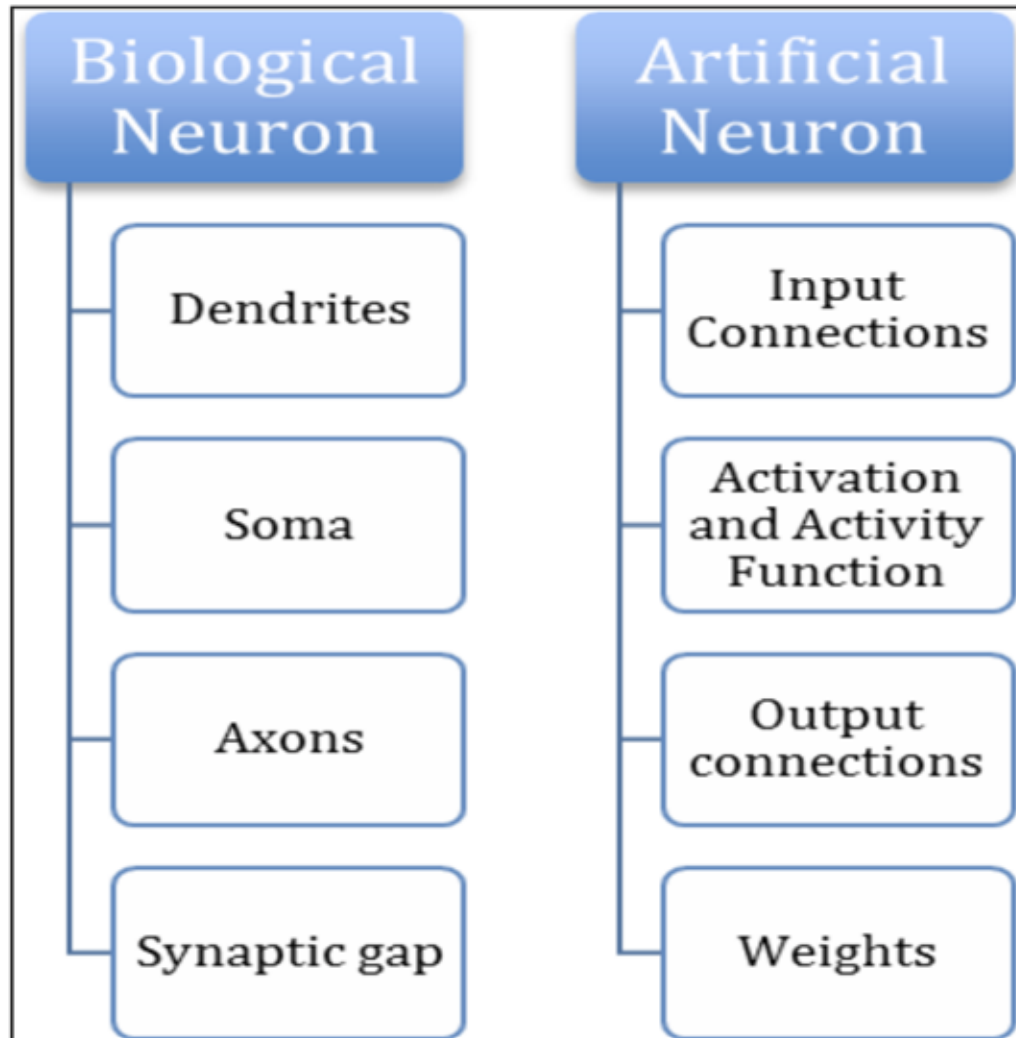
In an artificial neural net, we model the frequency through a numerical weight: the higher the frequency, the higher the impulse and, therefore, the higher the weight.

Reference:

APA 7th Edition (American Psychological Assoc.)

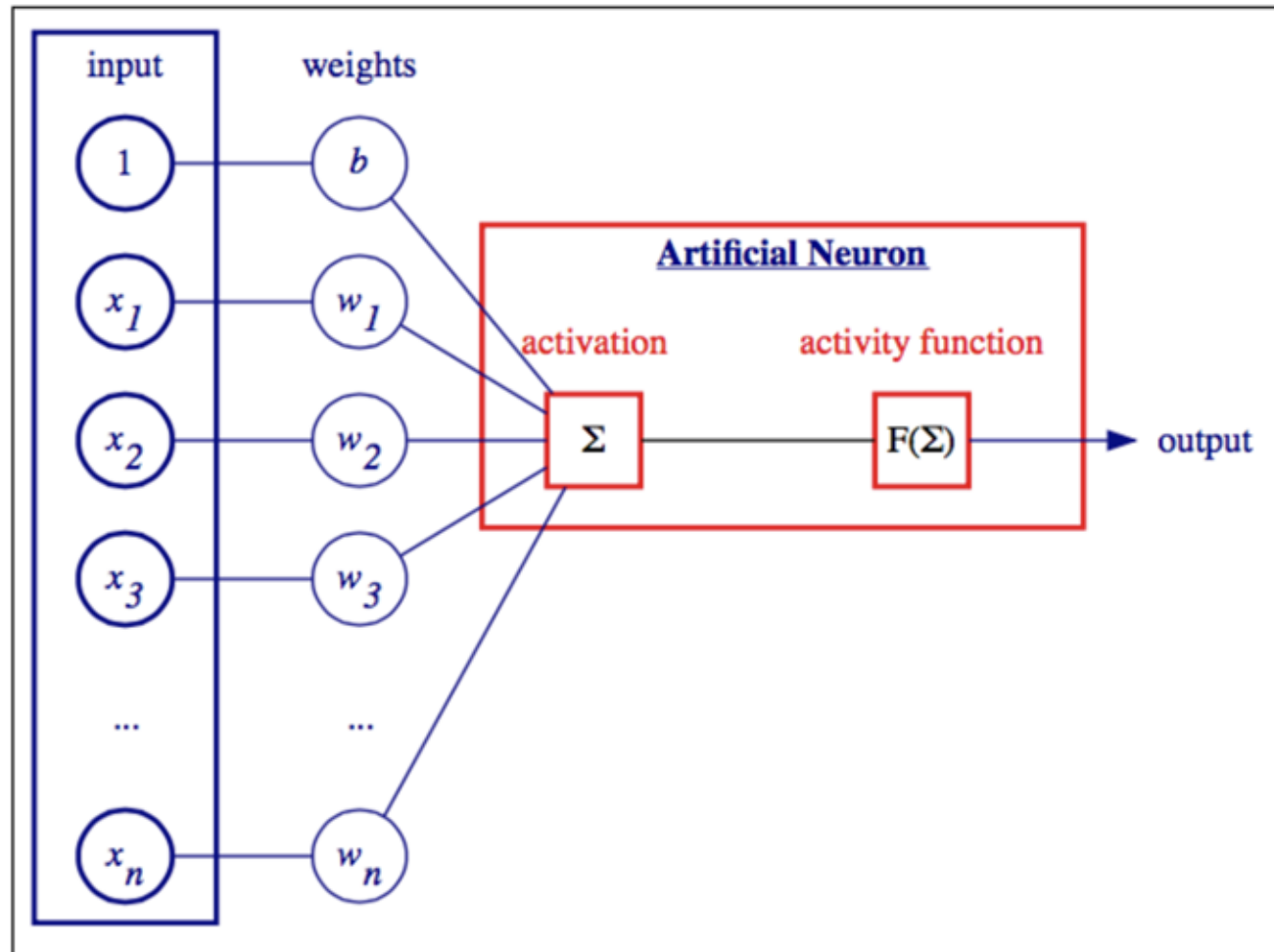
Zocca, V., Spacagna, G., Slater, D., & Roelants, P. (2017). Python Deep Learning. Packt Publishing.

Artificial Neural Networks



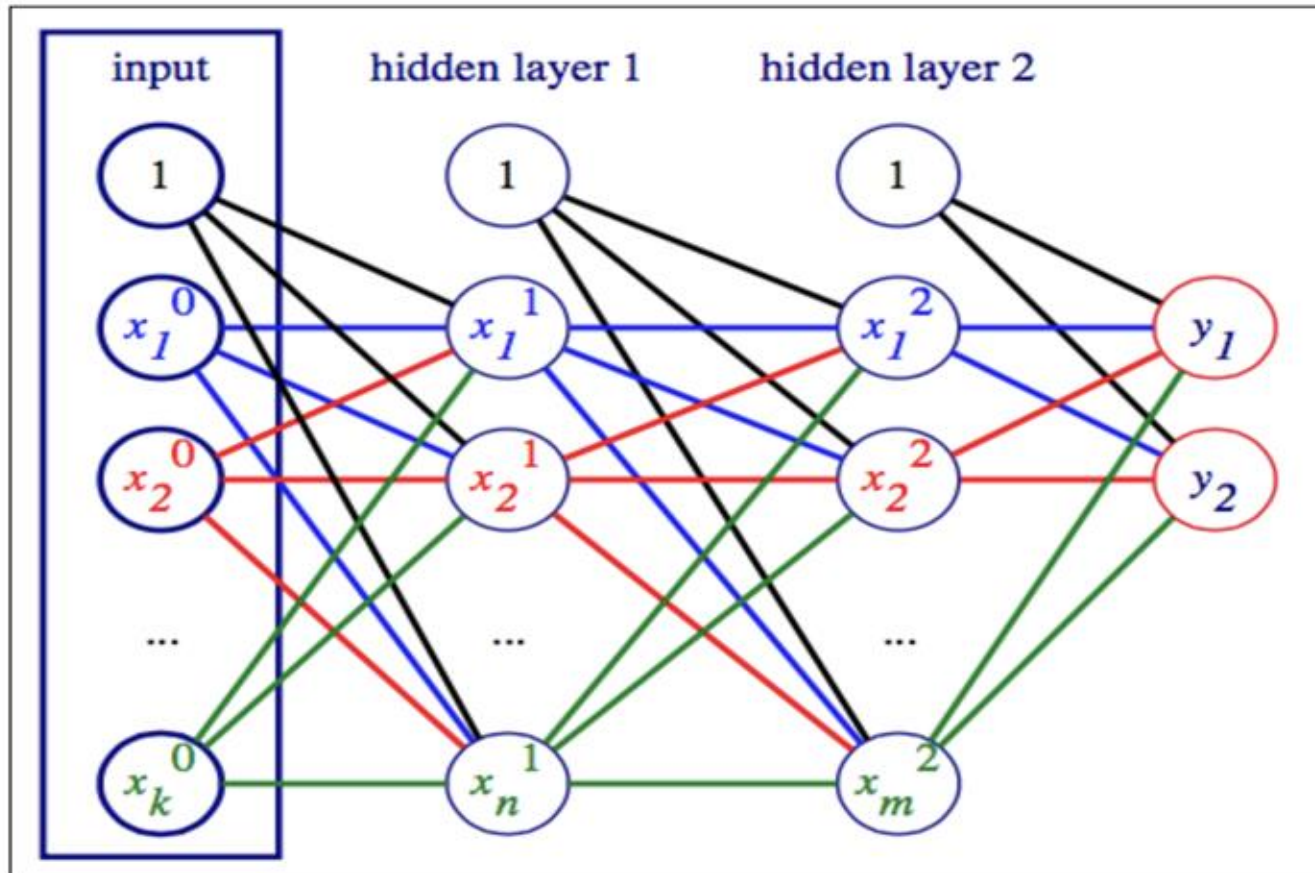
Schematic correspondence between a biological and an artificial neuron

Layers



At the center of this picture we have the neuron, or the soma, which gets an input (the activation) and sets the neuron's internal state that triggers an output (the activity function). The input comes from other neurons and it is mediated in intensity by the weights (the synaptic gap).

Con't



Shown is a 3-layer neural network with two hidden layers. The input layer has k input neurons, the first hidden layer has n hidden neurons, and the second hidden layer has m hidden neurons. In principle it is possible to have as many hidden layers as desired. The output, in this example, is the two classes, y_1 and y_2 . On top the on always-on bias neuron. Each connection has its own weight w (not depicted for simplicity).

Activation Functions

Biologically, neuroscience has identified hundreds, more than a thousand different types of neurons and therefore we should be able to model at least some different types of artificial neurons. This can be done by using different types of activity functions, that is, the function defined on the internal state of the neuron represented by the activation calculated on the input from all the input neurons.

The activity function is a function defined on $a(x)$ and it defines the output of the neuron.

Reference:

APA 7th Edition (American Psychological Assoc.)

Zocca, V., Spacagna, G., Slater, D., & Roelants, P. (2017). Python Deep Learning. Packt Publishing.

Types of Activation Functions

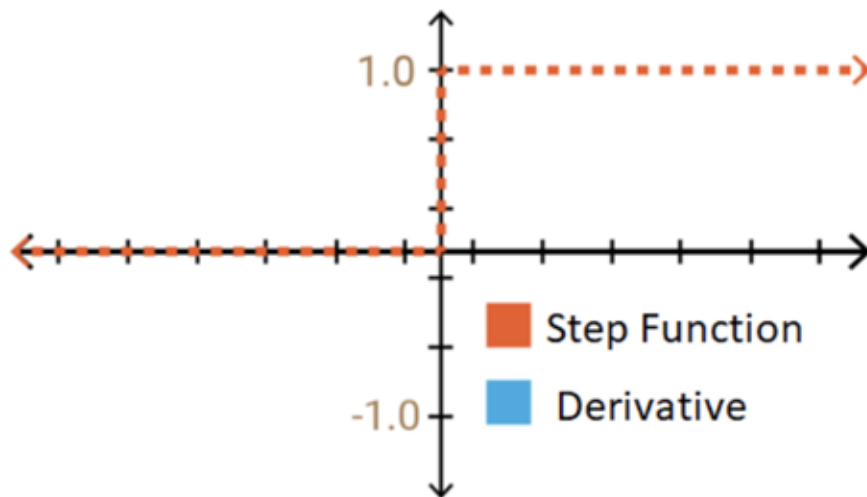
Binary Step Function

Outputs either 0 or 1.

$$f(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

Step Activation Function

Step Function



Step Function and Derivative

It is a function that takes a binary value and is used as a binary classifier. Therefore, it is generally preferred in the output layers. It is not recommended to use it in hidden layers because it does not represent derivative learning value and it will not appear in the future. Then, let's

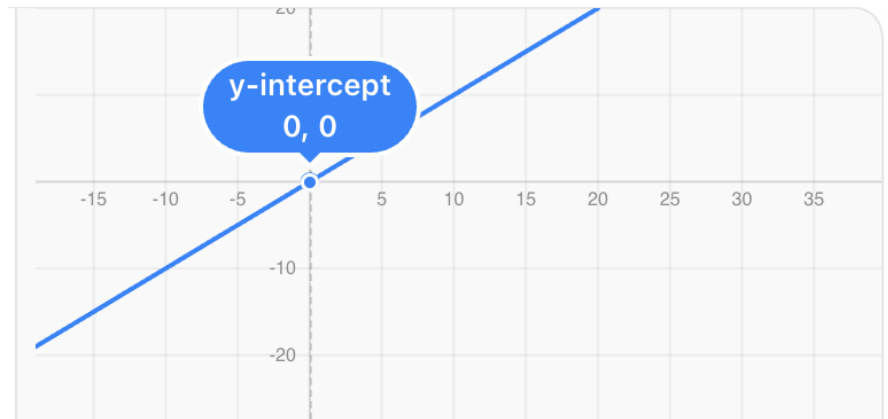
think of a derivative function, the linear function comes to mind immediately.

Types of Activation Functions

Linear Activation Function

Output is directly proportional to input.

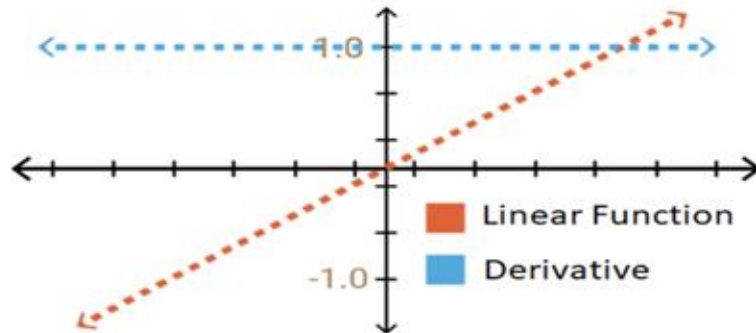
$$f(x) = x$$



- Used mostly in regression output layers
- Cannot model complex relationships alone

Linear Activation Function

Linear Function



Linear Function and Derivative

It generates a series of activation values and these are not binary values, as in the step function. It certainly allows you to connect several neurons (nerve cells) together. But this function has a major problem! Fixed the derivative. **So why do we need its derivative and what is the negativity of it**

being fixed? What we said; with the backpropagation algorithm, we performed the learning process for neurons. This algorithm consists of a derivative system. When $A = c \cdot x$ is derived from x , we reach c . This means that there is no relationship with x . Well, if the derivative is always a constant value, can we say that the learning process is taking place? Unfortunately no!

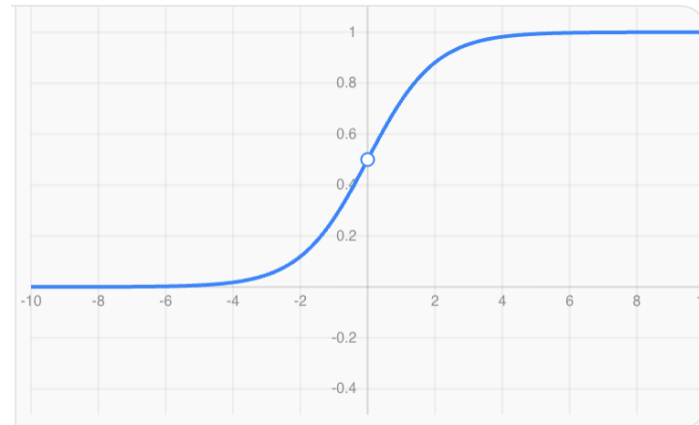
There is another problem! When the linear function is used in all layers, the same linear result is reached between the input layer and the output layer. **The linear combination of linear functions is another linear function.** This means that the neurons that we call at the very beginning can interfere with the interconnecting layers! 🤔

Types of Activation Functions

Sigmoid Function

Maps values between 0 and 1.

$$f(x) = \frac{1}{1+e^{-x}}$$



Features

- Smooth curve
- Useful for binary classification

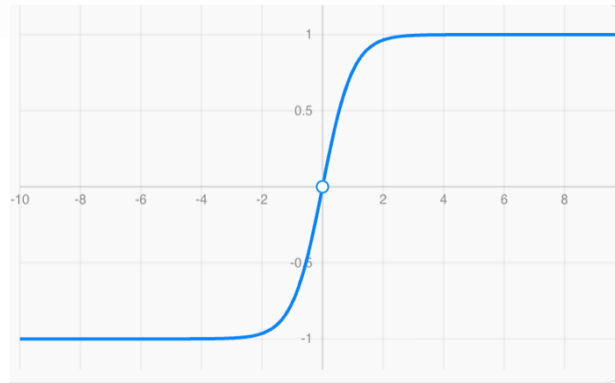
--

Types of Activation Functions

Tanh (Hyperbolic Tangent)

Maps values between -1 and 1.

$$f(x) = \tanh(x)$$

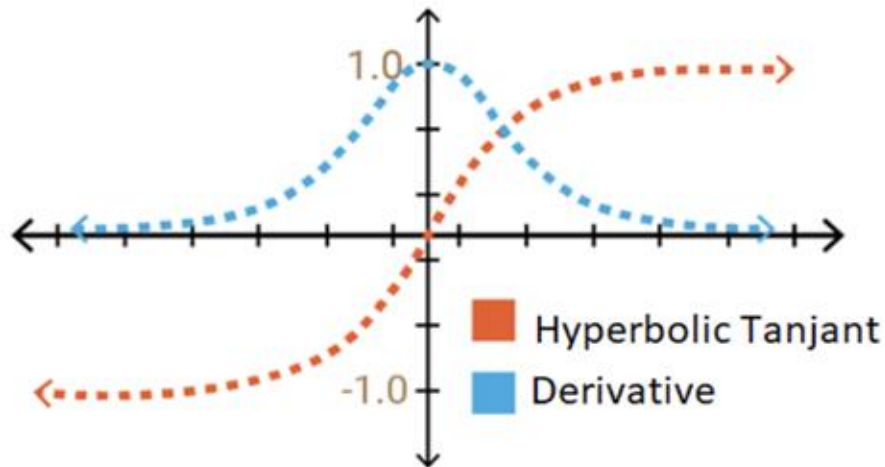


Features

- Zero-centered
- Better than sigmoid in many cases

Hyperbolic Tangent Activation Function

Hyperbolic Tangent Function



Hyperbolic Tangent and Derivative

It has a structure very similar to Sigmoid function. However, this time the function is defined as $(-1, +1)$. The advantage over the sigmoid function is that its derivative is more steep, which means it can get more value. This means that it will be more efficient because it has a wider range for faster learning and

grading. But again, the problem of gradients at the ends of the function continues. Although we have a very common activation function, we will continue our search to find the best one!

Types of Activation Functions

ReLU (Rectified Linear Unit)

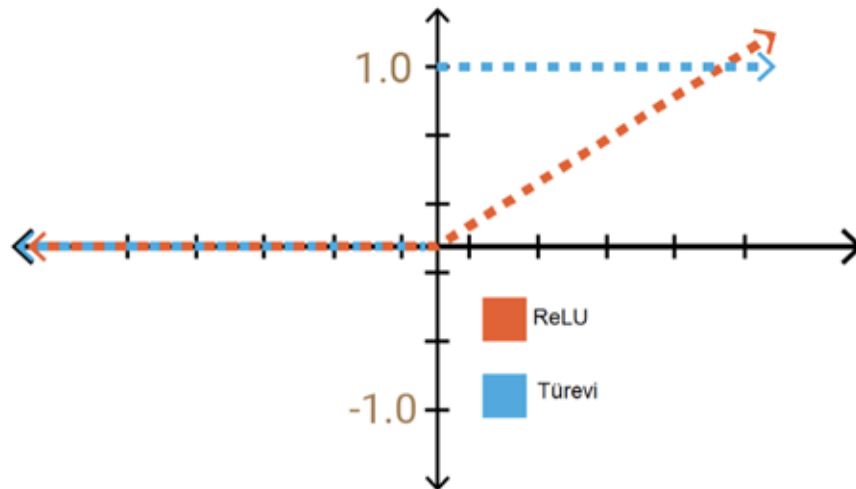
Most widely used activation in deep learning.

Features

- Fast and simple
- Helps deep networks train efficiently

ReLU Activation Function

ReLU (Rectified Linear Unit) Function



ReLU Function and Derivative

At first glance, it will appear to have the same characteristics as the linear function on the positive axis. But above all, ReLU is not linear in nature. In fact, a good estimator. It is also possible to converge with any other function by combinations of ReLU. Great! That means we can still sort the layers in our

artificial neural network (again) 😊

Types of Activation Functions

■ ELU (Exponential Linear Unit)

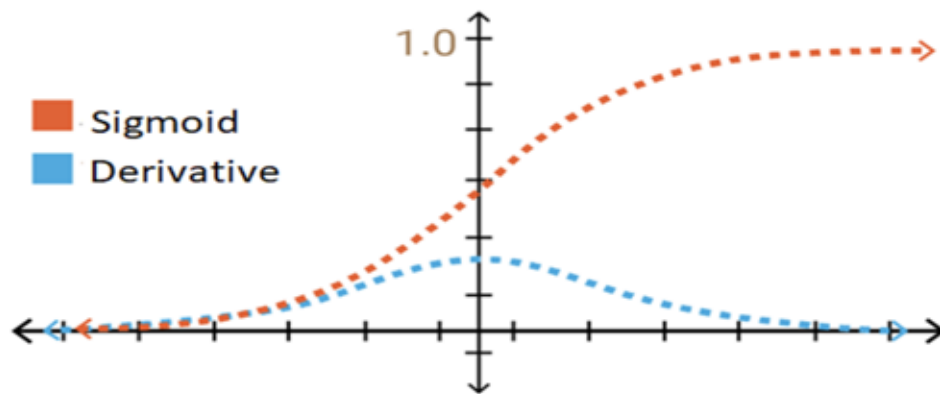
$$f(x) = \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases}$$

- Faster learning
- Handles negative values better than ReLU

Sigmoid Activation Function

Sigmoid Function

Imagine that most problems in nature are not linear and combinations of sigmoid function are not linear. Bingo!



Sigmoid Function and Derivative

Then we can sort the layers 😊
So let's think of non-binary functions. It is also derivated because it is different from the step function. This means that learning can happen. If we examine the graph x is between -2 and +2, y values change quickly. Small changes in x will be large in y. This means that it can be used as a good classifier.

Another advantage of this function is that it produces a value in the range of (0,1) when encountered with (- infinite, + infinite) as in the linear function. So the activation value does not vanish, this is good news! 🍷

The sigmoid function is the most frequently used activation function, but there are many other and more efficient alternatives.

Types of Activation Functions

■ Softmax Function

Used in multi-class classification.

$$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

- Converts outputs into probabilities
- Output values sum to 1

Which Activation Function is the best

ReLU → Most common for hidden layers

Sigmoid → Binary classification output

Softmax → Multi-class classification output

Linear → Regression tasks

Deep Learning Applications

- Speech recognition
- Object recognition and classification

GPU versus CPU

One of the reasons for the popularity of deep learning today is the drastically increased processing capacity of GPUs (Graphical Processing Units). Architecturally, the CPU (Central Processing Unit) is composed of a few cores that can handle a few threads at a time, while GPUs are composed of hundreds of cores that can handle thousands of threads at the same time. A GPU is a highly parallelizable unit, compared to the CPU that is mainly a serial unit.

DNNs are composed of several layers, and each layer has neurons that behave in the same manner. All activation values are calculated in the same way across the network. CPUs and GPUs have a different architecture, in particular they are optimized differently: CPUs are latency optimized and GPUs are bandwidth optimized. In a deep neural network with many layers and a large number of neurons, bandwidth becomes the bottleneck, not latency, and this is the reason why GPUs perform so much better. In addition, the L1 cache of the GPU is much faster than the L1 cache for the CPU and is also larger.

Reference:

APA 7th Edition (American Psychological Assoc.)
Zocca, V., Spacagna, G., Slater, D., & Roelants, P. (2017). Python Deep Learning. Packt Publishing.

GPU versus CPU

CPU vs GPU at a Glance

Feature	CPU	GPU
Number of cores	Few (typically 4–64)	Hundreds to thousands
Core complexity	Powerful, optimized for sequential tasks	Simpler, optimized for parallel tasks
Best for	General-purpose computing	Large-scale matrix and vector operations
Deep learning training	Usually slower	Much faster
Power consumption	Lower	Higher
Cost	Usually included in systems	Additional hardware cost

GPU versus CPU

Multiplying two large matrices involves millions or billions of independent calculations that can be performed simultaneously.

A CPU might use a small number of powerful cores:

Core 1 → Task A

Core 2 → Task B

Core 3 → Task C

Core 4 → Task D

GPU versus CPU

A GPU uses thousands of smaller cores:

Core 1 → Calculation 1

Core 2 → Calculation 2

...

Core 5000 → Calculation 5000

This massive parallelism makes GPUs ideal for deep learning workloads.

GPU versus CPU

Example: Training a Neural Network

Suppose you are training an image classification model.

On a CPU

Processes data with limited parallelism

Training may take hours or days

On a GPU

Thousands of image pixels and neural network operations are processed simultaneously

Training may finish in minutes or hours

Depending on the model, GPUs can be **10× to 100× faster** than CPUs.

GPU versus CPU

Role of CPU During Deep Learning

Even when a GPU is used, the CPU still performs important tasks:

- Loading data from disk
- Data preprocessing
- Managing training processes
- Communication with the operating system
- Coordinating GPU execution
- A typical workflow looks like:

Dataset → CPU preprocessing → GPU training →
Results

GPU versus CPU

Deep Learning Framework Support

Popular frameworks automatically use GPUs when available:

- TensorFlow
- PyTorch

GPU versus CPU

Why NVIDIA GPUs Are Common

Many deep learning tools are optimized for:

- NVIDIA's CUDA platform
- cuDNN libraries
- Tensor Cores for AI acceleration

These technologies significantly speed up training.

GPU versus CPU

CPU vs GPU for Different Tasks

Task	CPU	GPU
Data preprocessing	✓ Excellent	✗ Usually not used
Small ML models	✓ Good	Not necessary
Deep neural network training	Possible but slow	✓ Preferred
Large language models	Very slow	✓ Essential
Real-time inference (small models)	✓ Good	Sometimes overkill
Computer vision training	Slow	✓ Excellent



This Unit 1 assignment introduces you to building and comparing predictive models using a real-world healthcare scenario. In a guided Google Colab lab, you will prepare appointment data, train a baseline logistic regression model, and build a small neural network using PyTorch. You will evaluate both models using common classification metrics and visualize their performance. The assignment concludes with a decision memo where you recommend a model for hospital no-show prediction, balancing accuracy, interpretability, and operational risk.

Step-by-step student instructions

1. Open the provided notebook template in Google Colab.
2. Upload the dataset file (hospital_no_show_synthetic.csv) to the Colab session or mount Google Drive.
3. Run the Setup cell to import libraries.
4. Load the dataset and confirm the target column is `no\_show` (1 = no-show, 0 = showed).
5. Create two quick EDA plots: lead_days distribution + no-show rate by appointment_type.
6. Create a preprocessing pipeline: scale numeric features and one-hot encode categorical features.
7. Train a baseline Logistic Regression model and report: Accuracy, Precision, Recall, F1, ROC-AUC + confusion matrix.
8. Convert preprocessed matrices to PyTorch tensors and build a 2-hidden-layer neural network with dropout.
9. Train the neural network for 15 epochs with Adam optimizer and plot train vs validation loss.
10. Evaluate the neural network using the same metrics as the baseline and compare results in a table.
11. Answer the reflection questions in the notebook.

Software Requirements (Google Colab Only)

- You do not need to install Python locally for this assignment.
- Use Google Colab (free) in your web browser with a Google account.

Getting Started in Colab

- Go to Google Colab and choose New Notebook.
- Rename your notebook and confirm the runtime is Python 3 (default).
- Upload your data files via the folder icon → Upload (or mount Google Drive if preferred).

Open notebook

Examples >

Recent >

Google Drive >

GitHub >

Upload >

Search notebooks



Title

Last opened ↓

First opened ↑↓



Welcome To Colab

8:58 PM

8:58 PM



+ New notebook

Cancel



Files



bin
Upload to session storage

boot

content

datalab

dev

etc

home

kaggle

lib

lib32

lib64

libx32

media

Disk

87.40 GB available

[]



Start coding or generate with AI



Untitled0.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

🔍 Commands + Code ▾ + Text | ▶ Run all ▾



Files



bin

Upload to session storage

boot

content

datalab

dev

etc

home

kaggle

lib

lib32

lib64

libx32

media

Disk

87.40 GB available

[]



Start coding or generate with AI

<#>



In this assignment you will learn how a baseline machine learning model differs from a neural network and why those differences matter in practice. You will prepare tabular data for modeling by encoding categorical variables and scaling numeric features so the models can learn effectively. You will then train and evaluate both approaches using appropriate classification metrics to understand how well they perform. Finally, you will make a deployment recommendation that balances model accuracy with interpretability and operational risk.

Tools: Google Colab + Python, PyTorch, scikit-learn, pandas, matplotlib

Files provided:

- Dataset: [hospital_no_show_synthetic.csv](#) 
- Notebook template:
[IN403_Unit1_NoShow_Lab_Template.ipynb](#) 

- Dataset: [hospital_no_show_synthetic.csv](#) 
- Notebook template: [IN403_Unit1_NoShow_Lab_Template.ipynb](#)

Step-by-step student instructions

1. Open the provided link
2. Upload the dataset to your Colab session or manually enter the link
3. Run the Setup cell
4. Load the dataset and filter for no-shows (1 = no-show, 0 = show)
5. Create two quick EDA plots

Open Link in New Tab
Open Link in New Window
Open Link in Split View
Open Link in Incognito Window
Open Link as

Save Link As...

Copy Link Address

Copy

Copy Link to Highlight

Search Google for "IN403_Unit1_NoShow_Lab_Template.ipynb"

 Ask Gemini

Print...

Translate Selection to English

Inspect

Generate + Code + Markdown Run All ...

Select

IN403 – Unit 1 Lab: Predicting Hospital No-Shows (PyTorch + HuggingFace)

Goal: Build a baseline model and a small neural network to predict whether a patient will miss an appointment.

What you will submit

1. This completed notebook (exported as `.ipynb`)
2. A short decision memo ($\approx$ 1 page) answering:
 - Which model would you recommend to the hospital and why?
 - How do you balance accuracy vs interpretability vs operational risk?

Dataset

Use the provided file: `hospital_no_show_synthetic.csv` (uploaded with this lab).

0) Setup

Run this cell first to install/import packages.

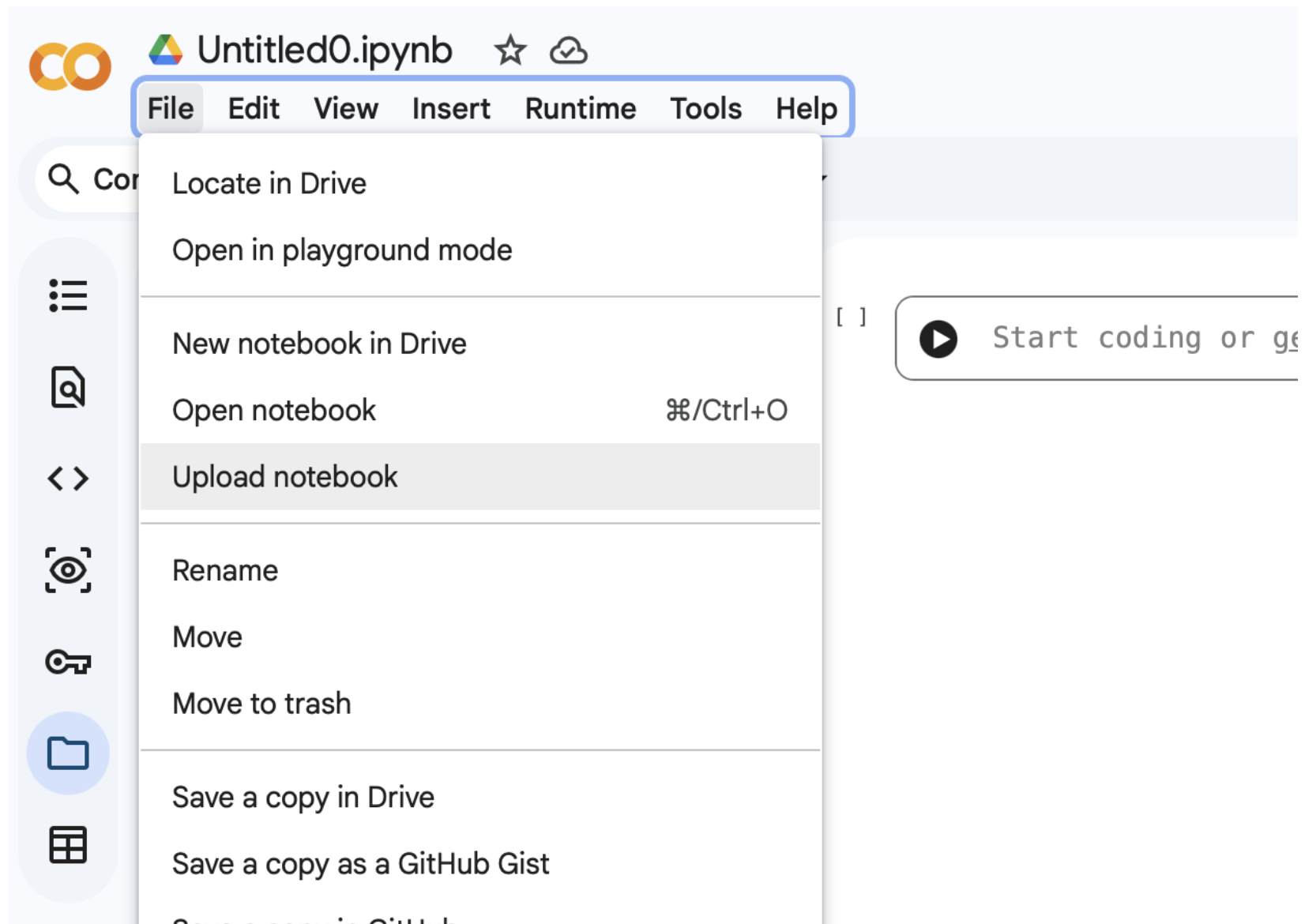
Generate

+ Code

+ Markdown

```
# If running in Colab, uncomment installs:
# !pip -q install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
# !pip -q install scikit-learn pandas matplotlib

import pandas as pd
import numpy as np
```





IN403_Unit1_NoShow_Lab_Template.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help



Share

An

Q Commands + Code + Text ▶ Run all

Connect ^



IN403 – Unit 1 Lab: Predicting Hospital No-Shows (PyTorch + HuggingFace)

Goal: Build a baseline model and a small neural network to predict whether a patient will miss an appointment.

What you will submit

1. This completed notebook (exported as `.ipynb`)
2. A short decision memo ($\approx$ 1 page) answering:
 - Which model would you recommend to the hospital and why?
 - How do you balance accuracy vs interpretability vs operational risk?

Dataset

Use the provided file: `hospital_no_show_synthetic.csv` (uploaded with this lab).

0) Setup

Run this cell first to install/import packages.



Share

An



RAM



Disk



download.pytorch.org/whl

_score, f1_score, roc_auc

Connect to a hosted runtime

Change runtime type

Connect to Google Cloud runtime

Connect to a local runtime

View resources

Manage sessions

Disconnect and delete runtime

Show executed code history

Focus the last run cell

- Dataset: [hospital_no_show_synthetic.csv](#)

- Notebook template:

IN403 Unit1 NoShow Lab Template

Step-by-step student instructions

1. Open the provided notebook template
2. Upload the dataset file (hospital_no_show_synthetic.csv) to the Colab session or mount Google Drive
3. Run the Setup cell to import libraries
4. Load the dataset and confirm the data types (1 = no-show, 0 = showed).

Open Link in New Tab

Open Link in New Window

Open Link in Split View

Open Link in Incognito Window

Open Link as



Save Link As...

Copy Link Address

Copy

Copy Link to Highlight

Search Google for "hospital_no_show_synthetic.csv"

 Ask Gemini

Print...

Translate Selection to English

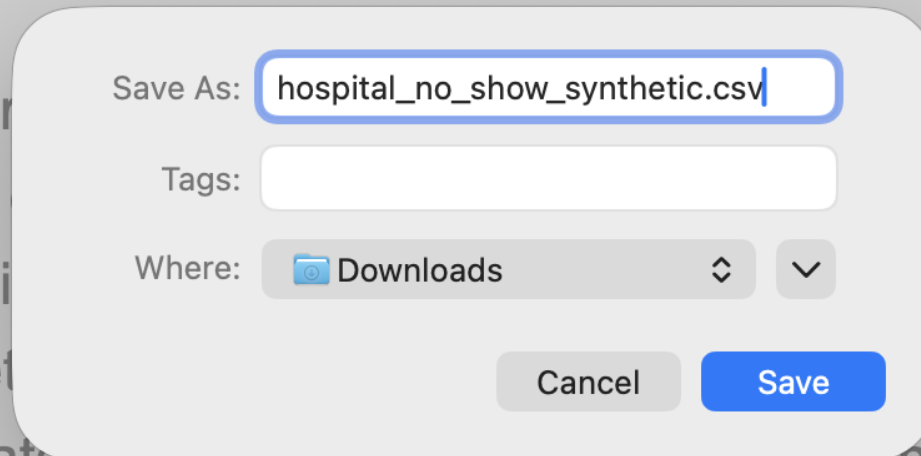
Inspect

If you save it using different name, make sure there is no space in the name, or replace space with _

- Dataset: [hospital_no_show_synthetic.csv](#) 
- Notebook template: [IN403_Unit1_NoShow_Lab_Template.ipynb](#) 

Step-by-step student instructions

1. Open the project in Google Colab.
2. Upload the dataset (hospital_no_show_synthetic.csv) to the Colab session.
3. Run the Setup cell.
4. Load the dataset and confirm the target column is 'no_show' (1 = no-show, 0 = showed).
5. Create two quick EDA plots: lead_days distribution + no-show rate by appointment type.



A save dialog box with a light gray background and rounded corners. It contains three input fields: 'Save As:' with the text 'hospital_no_show_synthetic.csv', 'Tags:' which is empty, and 'Where:' with a dropdown menu showing 'Downloads'. At the bottom right are two buttons: 'Cancel' and 'Save'.

Save As: hospital_no_show_synthetic.csv

Tags:

Where: Downloads

Cancel Save



Table data was imported and can be adjusted. >

hospital_no_show_synthetic

age	appointment_type	clinic	insurance_type	lead_days	prior_no_show_count	distance_km	day_of_week	hour_of_day	sms_opt_in	reminder_da
24	Primary Care	East	Medicaid	10	1	9.02	1	12	1	
73	Primary Care	South	Medicare	9	0	5.38	6	11	0	
65	Imaging	South	Medicare	22	0	4.06	6	16	1	
49	Lab	East	Private	22	0	5.23	0	9	1	
49	Primary Care	North	Medicaid	9	1	5.04	4	15	1	
79	Specialty	West	Medicare	10	0	2.16	2	12	1	
24	Specialty	East	Medicaid	19	0	15.04	1	11	0	
68	Primary Care	South	Medicaid	6	1	4.86	6	11	1	
32	Primary Care	South	Medicare	10	1	7.81	2	14	1	
24	Imaging	East	Self-pay	15	0	9.64	3	11	1	
55	Primary Care	North	Private	24	0	5.23	2	14	1	
88	Primary Care	West	Private	8	0	14.74	4	8	0	
70	Specialty	South	Private	0	0	10.76	1	10	1	
72	Primary Care	Central	Private	0	2	9.93	2	12	1	
69	Primary Care	East	Medicare	1	0	8.88	4	10	1	
74	Specialty	Central	Private	1	0	2.26	1	9	0	
54	Lab	Central	Medicare	12	0	3.74	5	10	0	
27	Imaging	Central	Self-pay	2	2	8.8	6	13	0	
78	Primary Care	Central	Medicaid	7	0	5.36	4	17	0	
50	Primary Care	South	Private	6	1	6.32	2	14	1	
54	Telehealth	South	Self-pay	10	3	7.03	1	17	1	
44	Imaging	West	Medicaid	27	0	10.0	3	13	1	



Q Commands + Code ▾ + Text | ▶ Run all ▾



Files



Upload to session storage



sample_data



README.md



anscombe.json



california_housing_test.csv



california_housing_train.csv



mnist_test.csv



mnist_train_small.csv

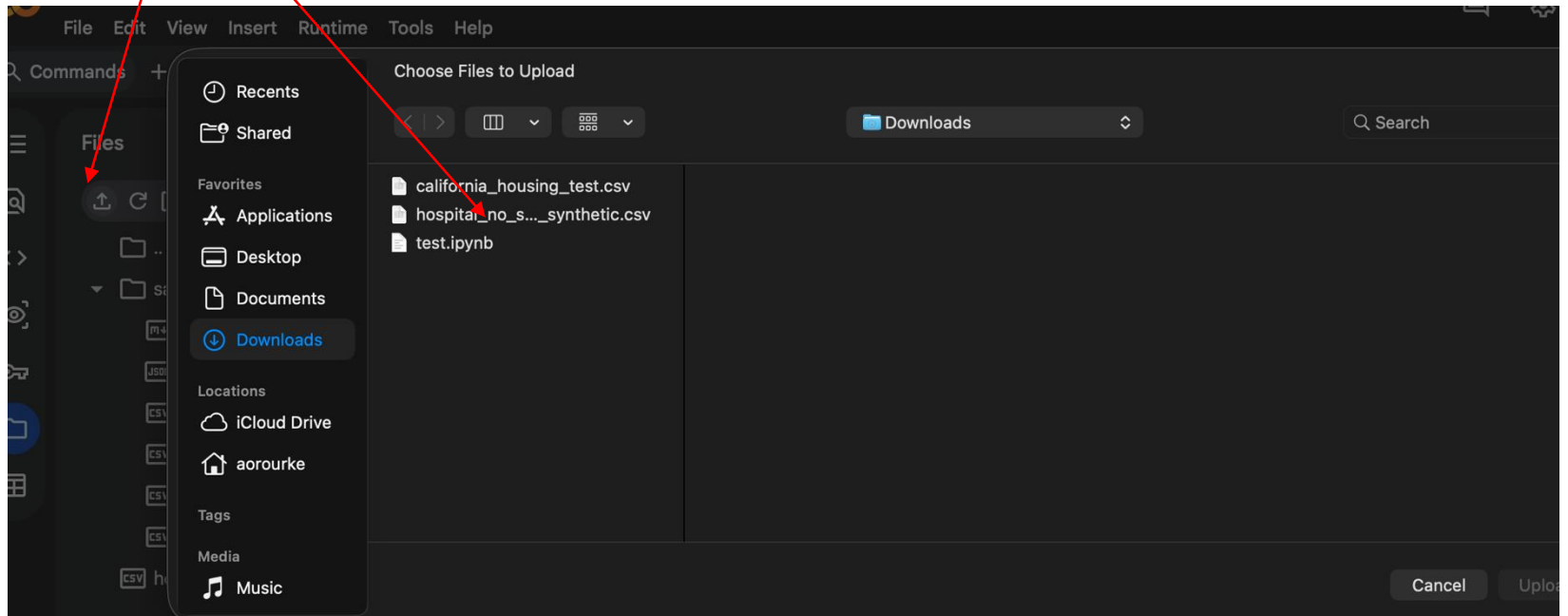
[1]

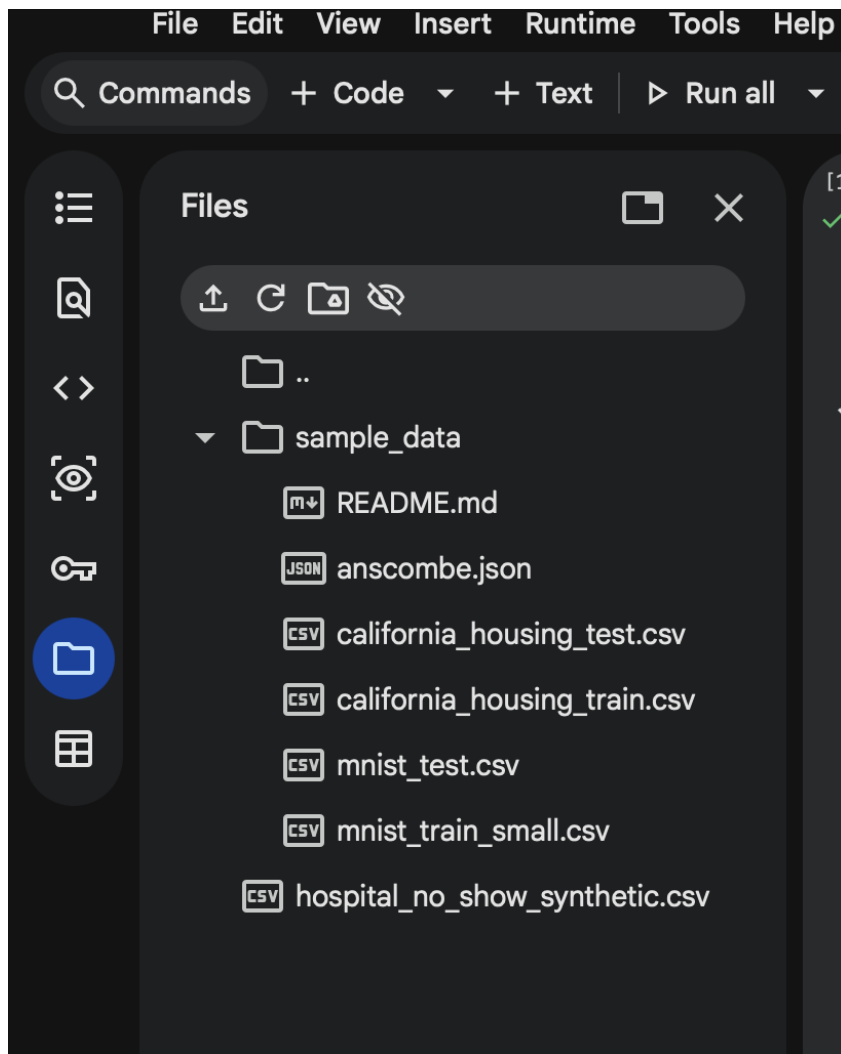
✓ 7s



[2]

! 0s





```
df = pd.read_csv('/content/hospital_no_show_synthetic.csv')
display(df.head())
print(df.shape)
```

```

display(df.head())
print(df.shape)
df.info()
print('\nMissing values:\n', df.isna().sum())
target_col = 'no_show'

```

... /content

	age	appointment_type	clinic	insurance_type	lead_days	prior_no_show_count	distance_km	day_of_week	hou
0	24	Primary Care	East	Medicaid	10	1	9.02	1	
1	73	Primary Care	South	Medicare	9	0	5.38	6	
2	65	Imaging	South	Medicare	22	0	4.06	6	
3	49	Lab	East	Private	22	0	5.23	0	
4	49	Primary Care	North	Medicaid	9	1	5.04	4	

(5000, 14)

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 5000 entries, 0 to 4999

Data columns (total 14 columns):

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	age	5000 non-null	int64
1	appointment_type	5000 non-null	object
2	clinic	5000 non-null	object
3	insurance_type	5000 non-null	object
4	lead_days	5000 non-null	int64

2) Quick EDA (Exploratory Data Analysis)

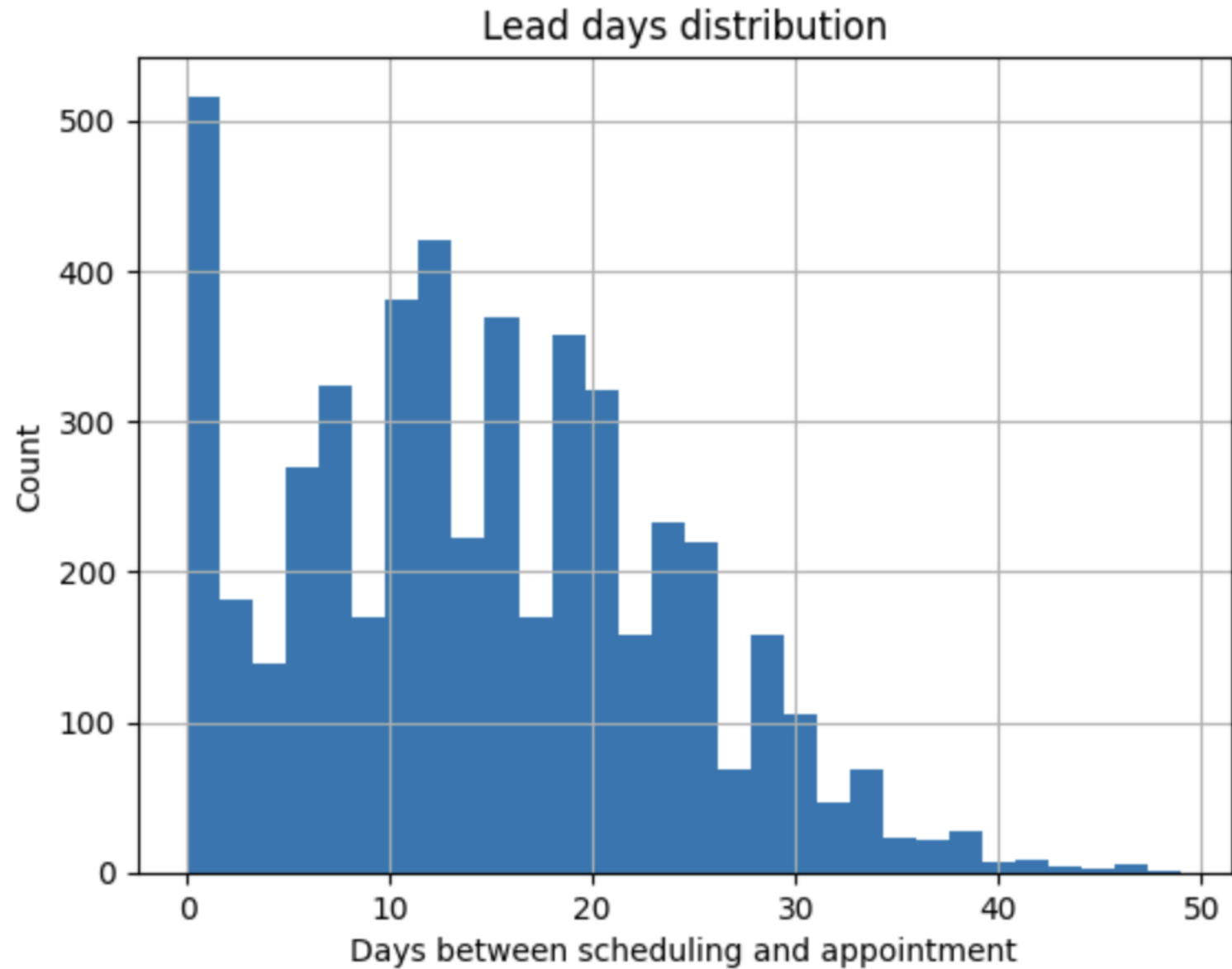
Student task: Create at least 2 plots:

- Distribution of `lead_days`
- No-show rate by appointment type (bar chart)



```
plt.figure()
df['lead_days'].hist(bins=30)
plt.title('Lead days distribution')
plt.xlabel('Days between scheduling and appointment')
plt.ylabel('Count')
plt.show()

plt.figure()
noshow_rate = df.groupby('appointment_type')[target_col].mean().sort_values(ascending=False)
noshow_rate.plot(kind='bar')
plt.title('No-show rate by appointment type')
plt.ylabel('No-show rate')
plt.show()
```



- Split into train/test sets
- Identify categorical vs numeric columns
- Build a preprocessing pipeline using `ColumnTransformer`

```
▶ X = df.drop(columns=[target_col])
  y = df[target_col].astype(int)

  cat_cols = ['appointment_type', 'clinic', 'insurance_type']
  num_cols = [c for c in X.columns if c not in cat_cols]

  X_train, X_test, y_train, y_test = train_test_split(
      X, y, test_size=0.2, random_state=SEED, stratify=y
  )

  preprocess = ColumnTransformer(
      transformers=[
          ('num', StandardScaler(), num_cols),
          ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols)
      ]
  )
```

(variable) cat_cols: list[str]

Final result

The pipeline will:

- Take raw data.
- Scale numeric columns.
- One-hot encode categorical columns.
- Produce a clean numerical matrix that a machine learning model can use.

This is a standard preprocessing setup before fitting models such as:

Logistic Regression

- Random Forest
- Gradient Boosting
- XGBoost
- Neural Networks

✓ 4) Baseline model: Logistic Regression

Student task:

- Train logistic regression
- Report Accuracy, Precision, Recall, F1, ROC-AUC
- Show a confusion matrix

Create the ML pipeline

A Pipeline chains multiple steps together.

Step 1: Preprocessing

The ColumnTransformer defined earlier:

- Numerical columns → standardized with StandardScaler
- Categorical columns → converted with OneHotEncoder

Step 2: Model

Uses Logistic Regression for binary classification.

Train the Model

```
baseline.fit(X_train, y_train)
```

Make the prediction

```
pred = baseline.predict(X_test)
```

Predicted Probabilities

```
proba = baseline.predict_proba(X_test)[: , 1]
```

predict_proba() returns two columns:

```
[[0.90 0.10],  
 [0.20 0.80],  
 [0.65 0.35]]
```

Column meanings:

- Column 0 → probability of class 0
- Column 1 → probability of class 1

`[:,1]` extracts only the probability of the positive class.

```
[0.10, 0.80, 0.35]
```

Calculate evaluation metrics

```
metrics = {  
    'accuracy': accuracy_score(y_test, pred),  
    'precision': precision_score(y_test, pred),  
    'recall': recall_score(y_test, pred),  
    'f1': f1_score(y_test, pred),  
    'roc_auc': roc_auc_score(y_test, proba)  
}
```

Accuracy

```
accuracy_score(y_test, pred)
```

Percentage of all predictions that were correct.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

```
precision_score(y_test, pred)
```

Of all predicted positives, how many were actually positive?

$$Precision = \frac{TP}{TP + FP}$$

High precision means few false alarms.

Recall:

```
recall_score(y_test, pred)
```

Of all actual positives, how many were found?

$$Recall = \frac{TP}{TP + FN}$$

High recall means few missed positives.

:

F1 Score:

```
f1_score(y_test, pred)
```

Balances precision and recall.

$$F1 = 2 \frac{Precision \times Recall}{Precision + Recall}$$

:

ROC-AUC

```
roc_auc_score(y_test, proba)
```

Measures how well the model separates the classes across all thresholds.

ROC-AUC	Meaning
0.5	Random guessing
0.6–0.7	Weak
0.7–0.8	Fair
0.8–0.9	Good
>0.9	Excellent

Print the metrics

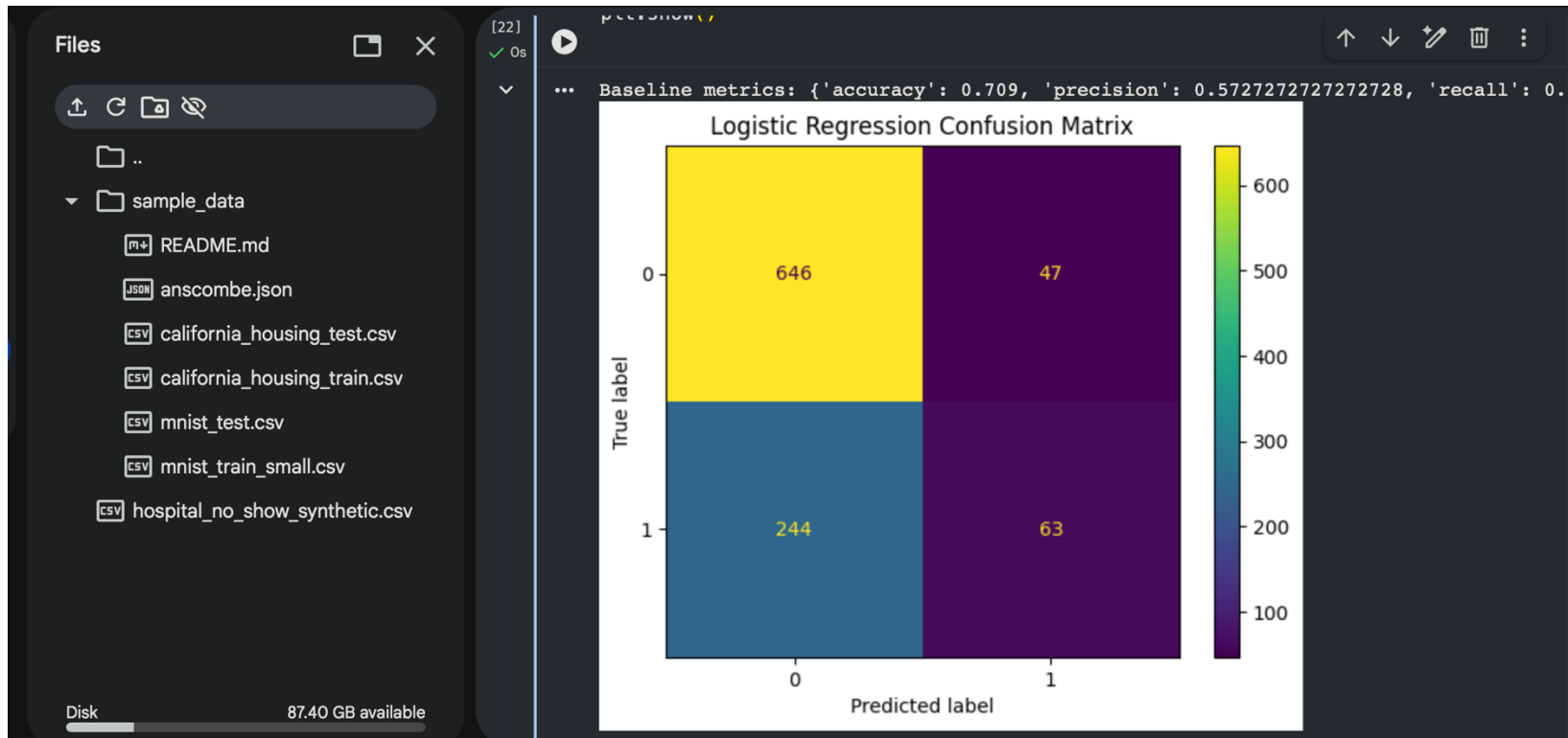
```
Baseline metrics: {  
  'accuracy': 0.84,  
  'precision': 0.79,  
  'recall': 0.72,  
  'f1': 0.75,  
  'roc_auc': 0.88  
}
```

Build a confusion matrix

```
cm = confusion_matrix(y_test, pred)
```

	Predicted 0	Predicted 1
Actual 0	TN	FP
Actual 1	FN	TP

```
[[150, 10],  
 [ 20, 70]]
```



5) Prepare tensors for PyTorch

Student task:

- Fit the preprocessing transformer on training data
- Transform train/test features into numeric matrices
- Convert to PyTorch tensors

```
# Fit transformer on training data only
X_train_p = preprocess.fit_transform(X_train)
X_test_p = preprocess.transform(X_test)

X_train_t = torch.tensor(X_train_p.toarray() if hasattr(X_train_p, 'toarray') else
X_test_t = torch.tensor(X_test_p.toarray() if hasattr(X_test_p, 'toarray') else
y_train_t = torch.tensor(y_train.values, dtype=torch.float32).view(-1, 1)
y_test_t = torch.tensor(y_test.values, dtype=torch.float32).view(-1, 1)

X_train_t.shape, y_train_t.shape

... (torch.Size([4000, 24]), torch.Size([4000, 1]))
```

6) Build a simple neural network

Requirements:

- Feed-forward network
- 2 hidden layers
- ReLU activations
- Dropout (0.2)
- Output layer with Sigmoid (binary classification)

Student task: Complete the model class and confirm the output shape.

```
▶ input_dim = X_train_t.shape[1]

class NoShowNet(nn.Module):
    def __init__(self, input_dim: int):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(64, 64),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(64, 1),
            nn.Sigmoid()
```

7) Train the model

Student task:

- Use `BCELoss` (binary cross-entropy)
- Use Adam optimizer
- Train for 15 epochs
- Track training loss and validation loss
- Plot both curves

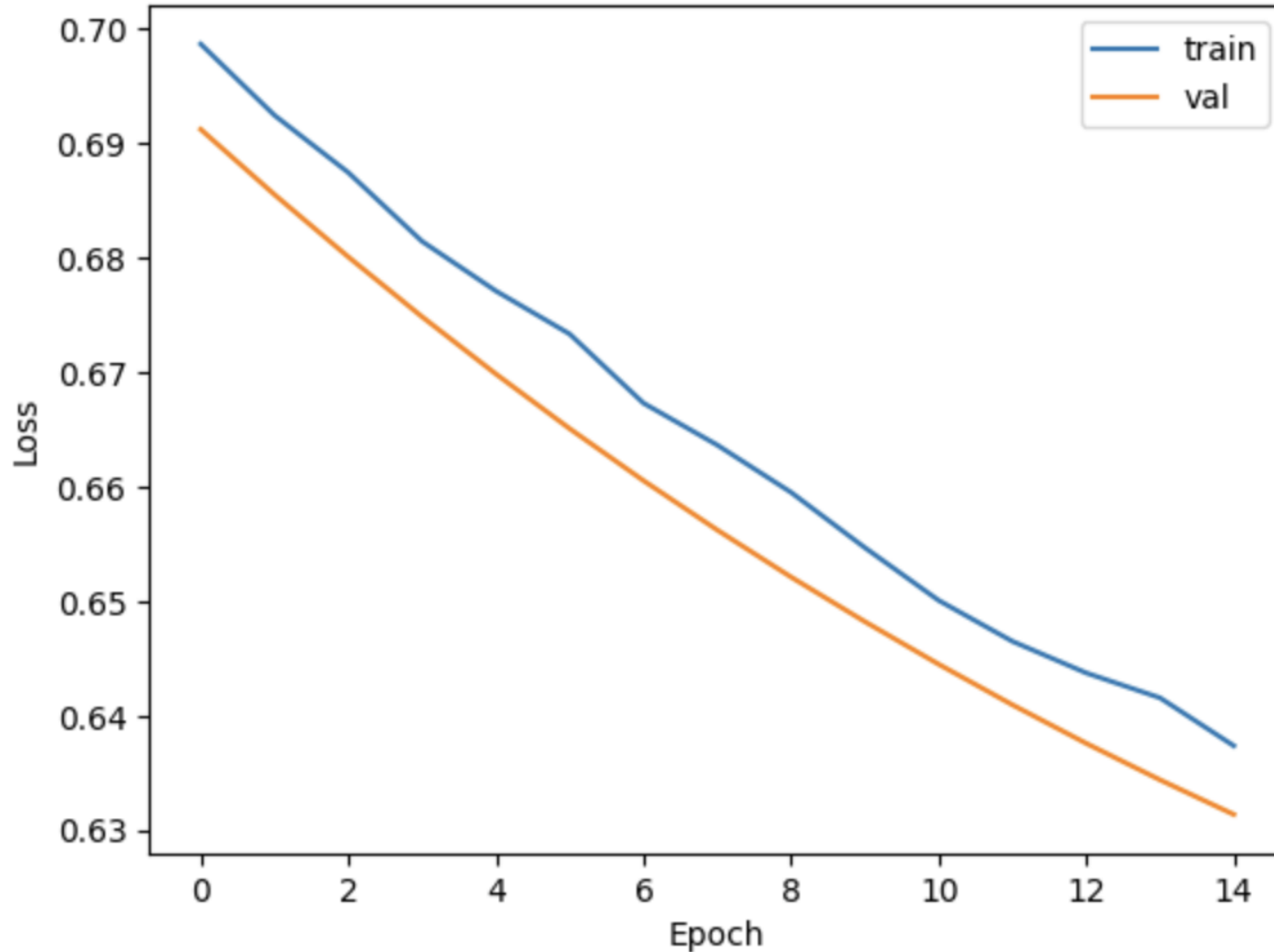
```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = model.to(device)

X_train_t_d = X_train_t.to(device)
y_train_t_d = y_train_t.to(device)
X_test_t_d = X_test_t.to(device)
y_test_t_d = y_test_t.to(device)

criterion = nn.BCELoss()
```

```
Epoch 5/15 | train_loss=0.6771 | val_loss=0.6698  
Epoch 10/15 | train_loss=0.6547 | val_loss=0.6482  
Epoch 15/15 | train_loss=0.6374 | val_loss=0.6314
```

Loss Curves



8) Evaluate the neural network

Student task:

- Compute Accuracy, Precision, Recall, F1, ROC-AUC
- Create a confusion matrix
- Compare results against logistic regression in a single table

```
model.eval()          Put the model in evaluation mode
with torch.no_grad():
    probs = model(X_test_t_d).cpu().numpy().ravel()

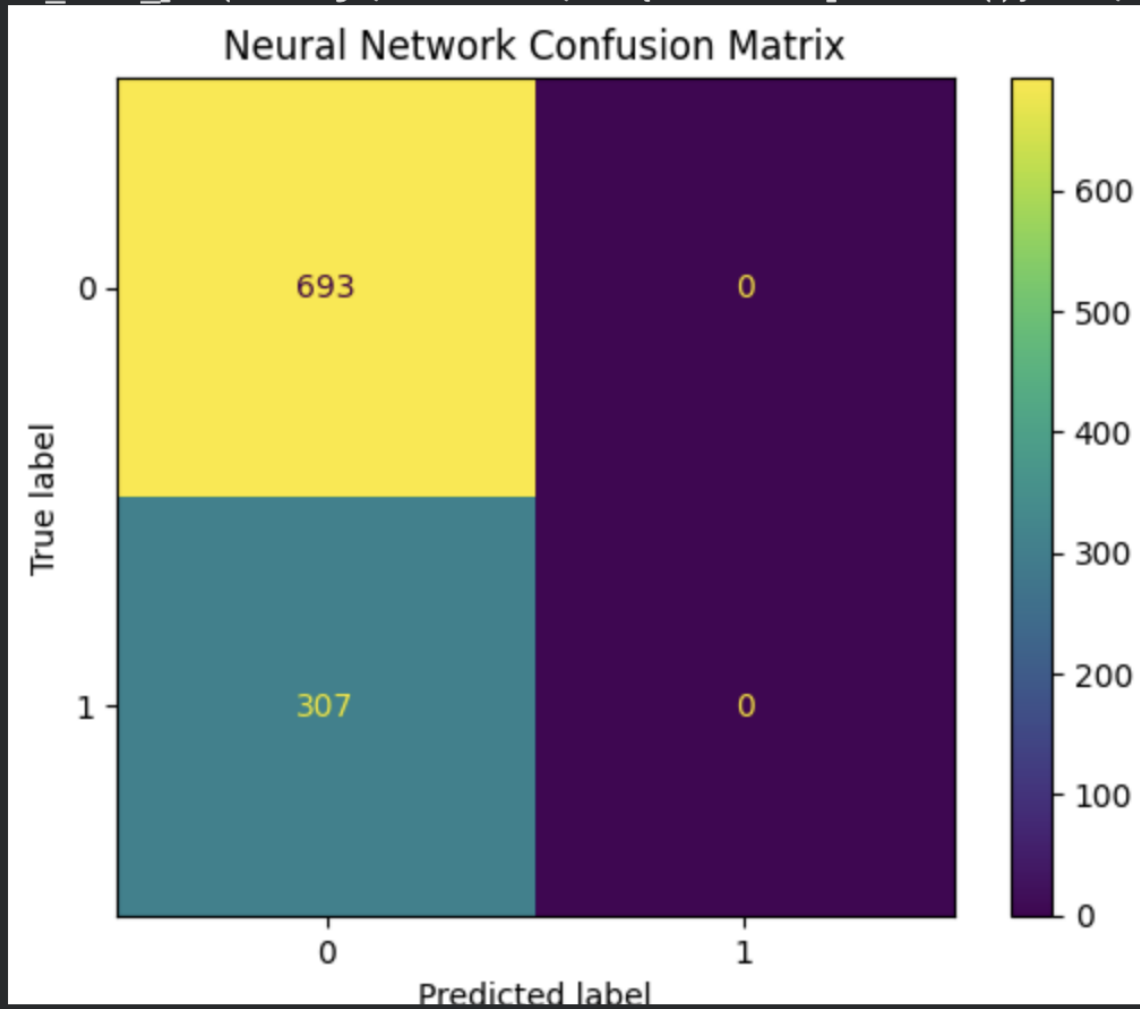
preds = (probs >= 0.5).astype(int)

nn_metrics = {
    'accuracy': accuracy_score(y_test, preds),
    'precision': precision_score(y_test, preds),
    'recall': recall_score(y_test, preds),
    'f1': f1_score(y_test, preds),
    'roc_auc': roc_auc_score(y_test, probs)
}

print('Neural net metrics:', nn_metrics)

cm = confusion_matrix(y_test, preds)
ConfusionMatrixDisplay(cm).plot()
plt.title('Neural Network Confusion Matrix')
plt.show()
```

```
Neural net metrics: {'accuracy': 0.693, 'precision': 0.0, 'recall': 0.0, 'f1': 0.0,  
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: Un  
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```



	model	accuracy	precision	recall	f1	roc_auc
0	Logistic Regression	0.709	0.572727	0.205212	0.302158	0.694154
1	Neural Network	0.693	0.000000	0.000000	0.000000	0.540200

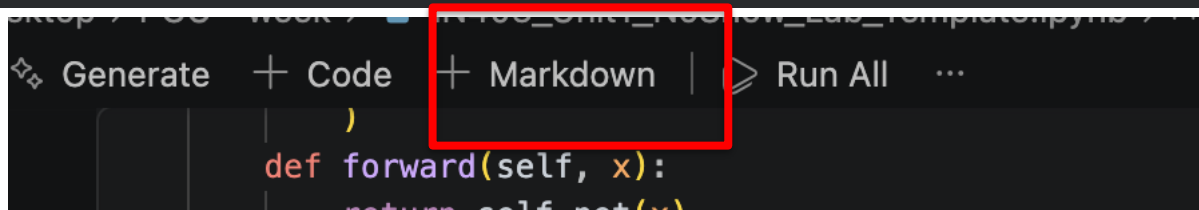


9) Reflection questions (answer in markdown)

1. Which model would you deploy at the hospital and why?
2. Did the neural network meaningfully outperform the baseline? If yes, *how much* improvement did you see?
3. What is one risk of using a black-box model in healthcare operations?
4. If the hospital demanded interpretability, what would you change?

Optional AI Extension (bonus)

Use a HuggingFace **tabular model** or explainability tool (e.g., SHAP) to explain feature importance and write 1 paragraph interpreting the top drivers of no-shows.



MEMO (Word document)

Write a short memo to the hospital executive team (about 300–500 words) that answers:

1. Which model do you recommend for initial deployment and why?
2. If you recommend the neural network, what controls would you add to reduce risk (monitoring, thresholding, explainability)?
3. If you recommend logistic regression, what might you lose in performance, and why is that acceptable?
4. What metric matters most in this operational setting—precision, recall, or something else? Explain.

Deliverables

DELIVERABLES

- Completed Colab notebook (.ipynb) with code cells executed and outputs visible.
- Two EDA plots.
- Loss curve plot (train vs validation).
- Confusion matrices for both models.
- Model comparison table (baseline vs neural net).
- Decision memo ($\approx$ 1 page) embedded at the end of the notebook in markdown

ASSIGNMENT REQUIREMENTS

Be sure to upload your Jupyter notebook (.ipynb) file with each cell annotated. Also, all written portions of the assignment should be in a markup cell.